NAME :- Lokare Mahir Abdul Majeed

ROLL NO :- 23AI34

BATCH :- 02

EXPERIMENT NO :- 07

AIM :- To design and implement a Convolutional Neural Network (CNN) model for handwritten digit recognition using the MNIST dataset.

OBJECTIVE :-

1. To understand the concept of Convolutional Neural Networks.
2. To understand convolution and pooling operations.
3. To design a CNN model for handwritten digit recognition.
4. To train the CNN using the MNIST dataset.
5. To evaluate the model using test accuracy and loss.
6. To visualize the predicted handwritten digits.

THEORY :-

1. Convolutional Neural Network :-

A Convolutional Neural Network (CNN) is a deep learning model mainly designed for processing image data. Unlike a fully connected neural network, a CNN preserves the spatial relationship between pixels and automatically learns important visual features such as edges, textures, shapes and patterns. CNNs are widely used in image classification and handwritten digit recognition.

For handwritten digit recognition, the MNIST dataset is commonly used. It contains grayscale images of handwritten digits from 0 to 9, where each image has a size of 28 × 28 pixels. The CNN learns the visual patterns of these digits during training and predicts the digit represented by a new image.

2. Main Components of CNN

A typical CNN consists of the following layers:

Input Layer:

The input layer receives the image. For MNIST, each image is represented as 28 × 28 × 1, where 1 represents the grayscale channel. Pixel values are normalized to the range 0 to 1 before training.

Convolutional Layer:

The convolutional layer applies small learnable filters or kernels to the input image. These filters move across the image and perform element-wise multiplication and summation to generate feature maps. Different filters learn different features such as edges, curves and textures.

A simplified convolution operation can be represented as:

$$S(i, j) = \Sigma m \, \Sigma n \, I(i+m, j+n) \, K(m, n)$$

where I is the input image and K is the convolution kernel.

ReLU Activation:

After convolution, the Rectified Linear Unit (ReLU) activation function is commonly applied. It introduces non-linearity into the network and allows it to learn complex patterns.

$$ReLU(x) = max(0, x)$$

Thus, negative values are converted to zero while positive values remain unchanged.

Pooling Layer:

Pooling reduces the spatial dimensions of feature maps while retaining important information. Max Pooling selects the maximum value from a small region. A common configuration is a 2 × 2 pool with stride 2, which reduces the height and width of the feature map. This reduces computation and the number of parameters.

Flatten Layer:

After convolution and pooling, the resulting feature maps are converted into a one-dimensional vector using the Flatten layer. This vector is then provided to the fully connected layers.

Fully Connected Layer:

The fully connected or dense layer uses the extracted features to perform classification. It combines the learned features and determines which digit the input image represents.

Output Layer:

For MNIST digit recognition, the output layer contains 10 neurons, corresponding to digits 0 through 9. A Softmax activation function converts the outputs into probabilities, and the class with the highest probability is selected as the predicted digit.

3. CNN Architecture for Handwritten Digit Recognition

The CNN implemented in this experiment follows the architecture: Input (28×28×1) → Conv2D (32 filters, 3×3, ReLU) → MaxPooling (2×2) → Conv2D (64 filters, 3×3, ReLU) → MaxPooling (2×2) → Flatten → Dense (64, ReLU) → Dense (10, Softmax).

The first convolutional layer learns simple features such as edges and curves. The second convolutional layer combines these features to learn more complex patterns. Pooling reduces the feature-map dimensions, while the dense layers use the extracted information for final classification. This general CNN structure is also commonly used for handwritten digit recognition with MNIST.

4. Working of CNN

The working process can be summarized as follows: the MNIST images are loaded and normalized; the images are reshaped to include a grayscale channel; convolutional filters extract important features from the images; ReLU introduces non-linearity into the network; max pooling reduces the spatial dimensions of the feature maps; further convolution extracts higher-level features; the Flatten layer converts the feature maps into a one-dimensional vector; dense layers process the extracted features; the Softmax output layer produces probabilities for the ten digit classes; the digit having the highest probability is selected as the predicted output.

During training, the model compares its predictions with the actual labels and calculates the loss. The optimizer updates the network weights through backpropagation so that the model gradually improves its classification performance.

PROCEDURE :-

1. Import TensorFlow and Matplotlib.
2. Load the MNIST handwritten digit dataset.
3. Normalize the pixel values between 0 and 1.
4. Reshape the images to include the channel dimension.
5. Create the CNN model.
6. Add the first convolutional layer with ReLU activation.
7. Add a Max Pooling layer.
8. Add the second convolutional layer with ReLU activation.
9. Add another Max Pooling layer.
10. Flatten the extracted feature maps.
11. Add a Dense layer with ReLU activation.
12. Add a 10-neuron Softmax output layer.
13. Compile the model using Adam optimizer and sparse categorical cross-entropy.
14. Train and evaluate the CNN model.
15. Display sample predictions and training accuracy.

CODE :-

```
import tensorflow as tf
import matplotlib.pyplot as plt

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
```

```
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

model = tf.keras.Sequential([
    tf.keras.Input(shape=(28, 28, 1)),
    tf.keras.layers.Conv2D(
        32,
        (3, 3),
        activation="relu"
    ),
    tf.keras.layers.MaxPooling2D((2, 2)),
    tf.keras.layers.Conv2D(
        64,
        (3, 3),
        activation="relu"
    ),
    tf.keras.layers.MaxPooling2D((2, 2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(
        64,
        activation="relu"
    ),
    tf.keras.layers.Dense(
        10,
        activation="softmax"
    )
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

history = model.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.2
)

test_loss, test_accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=0
)

print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)

predictions = model.predict(
    x_test[:10],
    verbose=0
)

predicted_classes = predictions.argmax(axis=1)

print("Predicted Classes:", predicted_classes)
print("Actual Classes:", y_test[:10])

plt.figure(figsize=(8, 5))

plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)
```

```
plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.title("CNN Training and Validation Accuracy")

plt.legend()
plt.grid(True)

plt.show()

plt.figure(figsize=(12, 4))

for i in range(10):

    plt.subplot(2, 5, i + 1)

    plt.imshow(
        x_test[i].squeeze(),
        cmap="gray"
    )

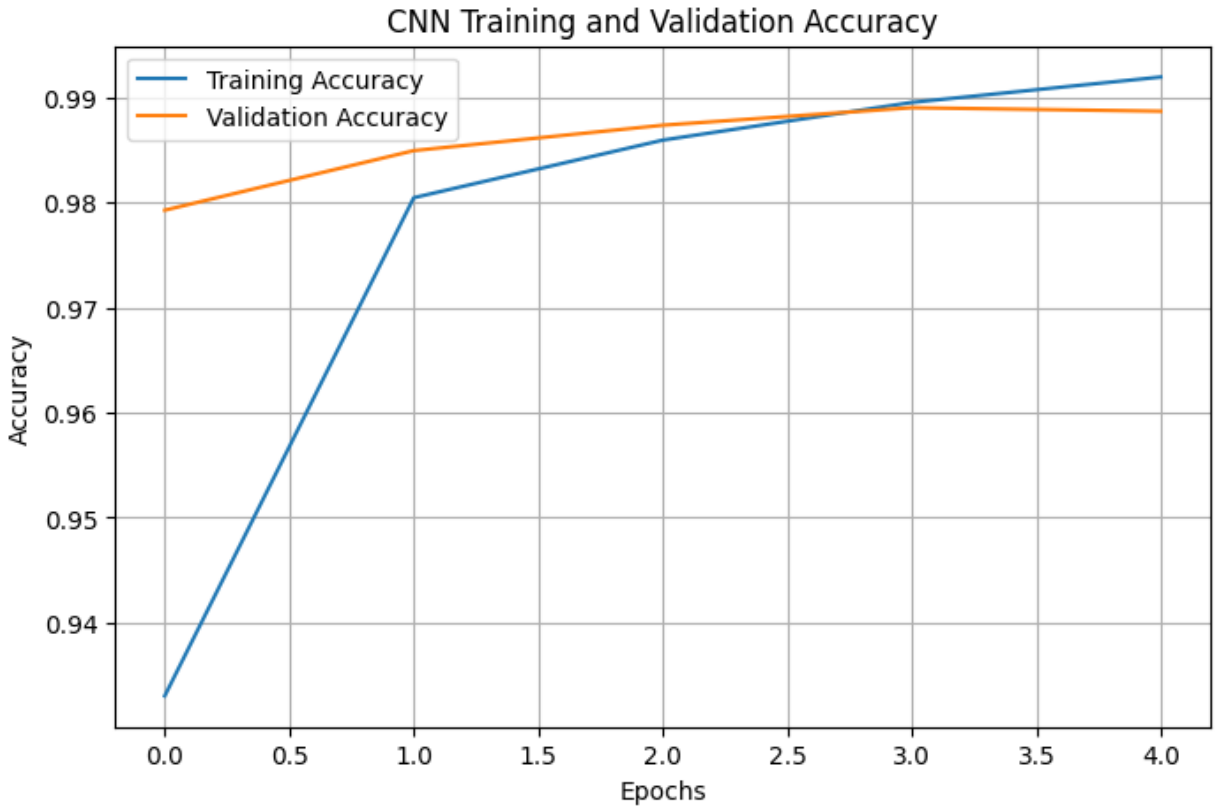
    plt.title(
        f"Pred: {predicted_classes[i]}"
    )

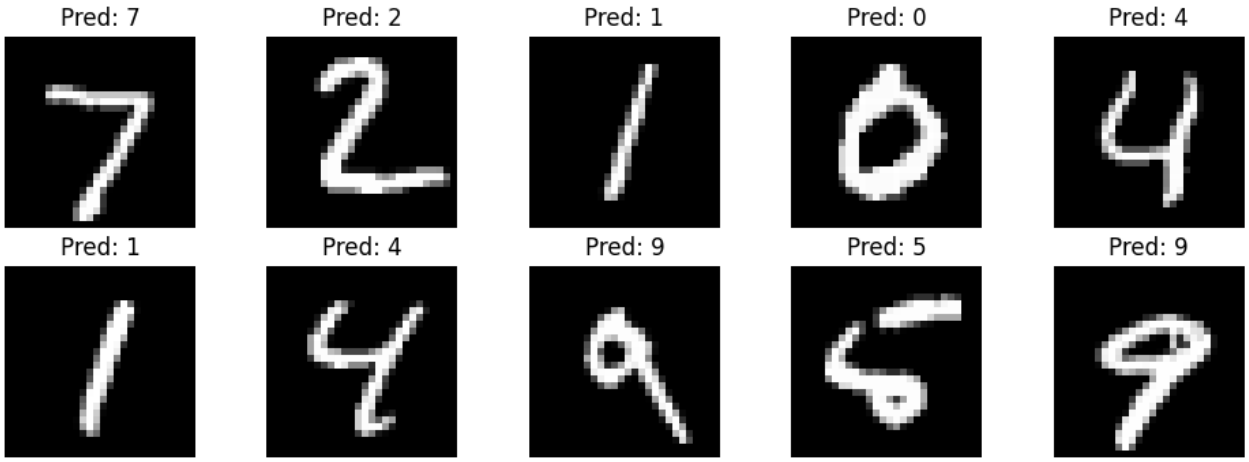
    plt.axis("off")

plt.show()
```

OUTPUT :-

```
Epoch 1/5
750/750 ━━━━━━━━━━━━━━━━━━━━ 28s 36ms/step - accuracy: 0.9331 - loss: 0.2251 - val_accuracy: 0.9793 - val_loss: 0.0721
Epoch 2/5
750/750 ━━━━━━━━━━━━━━━━━━━━ 39s 33ms/step - accuracy: 0.9804 - loss: 0.0633 - val_accuracy: 0.9849 - val_loss: 0.0509
Epoch 3/5
750/750 ━━━━━━━━━━━━━━━━━━━━ 26s 35ms/step - accuracy: 0.9859 - loss: 0.0454 - val_accuracy: 0.9873 - val_loss: 0.0431
Epoch 4/5
750/750 ━━━━━━━━━━━━━━━━━━━━ 25s 33ms/step - accuracy: 0.9895 - loss: 0.0344 - val_accuracy: 0.9890 - val_loss: 0.0394
Epoch 5/5
750/750 ━━━━━━━━━━━━━━━━━━━━ 41s 33ms/step - accuracy: 0.9919 - loss: 0.0262 - val_accuracy: 0.9887 - val_loss: 0.0403
WARNING:tensorflow:6 out of the last 6 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data
Test Loss: 0.03463669866323471
Test Accuracy: 0.9883999824523926
Predicted Classes: [7 2 1 0 4 1 4 9 5 9]
Actual Classes: [7 2 1 0 4 1 4 9 5 9]
```





CONCLUSION :-

The experiment successfully implemented a CNN for handwritten digit recognition using the MNIST dataset. The CNN learned image features through convolution and pooling layers and successfully classified handwritten digits into 10 classes.